

Tutorial 9

SC1006, CZ1106

Computer Arithmetic and Processor Pipeline

Numbers, numbers...

Ancient	Byzantine	Modern	Value	Ancient	Byzantine	Modern	Value	Ancient	Byzantine	Modern	Value	Ancient	Byzantine	Modern	Value
Α	ᾱ	Α'	1	Ι	ῐ	Ι'	10	Ρ	ῥ	Ρ'	100	Ϡ & ϡ	α	Α	1000
Β	β	Β'	2	Κ	κ	Κ'	20	Σ	σ	Σ'	200	Ϡ ^β	β	Β	2000
Γ	γ	Γ'	3	Λ	λ	Λ'	30	Τ	τ	Τ'	300	Ϡ ^γ	γ	Γ	3000
Δ	δ	Δ'	4	Μ	μ	Μ'	40	Υ	υ	Υ'	400	Ϡ ^Δ	Δ	Δ	4000
Ε	ε	Ε'	5	Ν	ν	Ν'	50	Φ	φ	Φ'	500	Ϡ ^Ε	ε	Ε	5000
Ζ Ζ	ζ & ς ζ & ϣ	Ζ' ΣΤ'	6	Ξ	ξ	Ξ'	60	Χ	χ	Χ'	600	Ϡ ^ζ	ζ & ς ζ & ϣ	Ζ ΣΤ	6000
Ζ	ζ	Ζ'	7	Ο	ο	Ο'	70	Ψ	ψ	Ψ'	700	Ϡ ^ι	ζ	Ζ	7000
Η	η	Η'	8	Π	π	Π'	80	Ω	ω	Ω'	800	Ϡ ^η	η	Η	8000
Θ	θ	Θ'	9	Ϟ ϙ	Ϟ & ϟ ϙ & Ϡ	ϙ'	90	Ϡ ϡ & Ϣ ϣ & Ϥ ϥ & Ϧ ϧ	Ϡ ϣ & ϣ ϣ & ϣ ϣ & ϣ ϣ	ϣ'	900	Ϡ ^θ	θ	Θ	9000

Numbers, numbers...

Roman numerals are a [numeral system](#) that was used by [ancient Rome](#). Numbers in this system use letters from the [Latin alphabet](#). Currently, it uses seven symbols:^[1]

Symbol	I	V	X	L	C	D	M
Value	1	5	10	50	100	500	1,000

It is very easy to write a number as a Roman numeral. Simply subtract the largest possible Roman numeral as many times as possible from the number. This system will result in a valid Roman numeral, but will not take the subtraction rule into account.

$$1 \times 1000 + 1 \times 500 + 4 \times 100 + 1 \times 50 + 3 \times 10 + 4 \times 1 = 1984$$

$$\mathbf{M} + \mathbf{D} + \mathbf{CCCC} + \mathbf{L} + \mathbf{XXX} + \mathbf{IIII} = \mathbf{MDCCCCLXXXIIII}$$

Getting the number from the numeral is equally simple, by adding the values of the symbols.

In general, the values for 5, 50, and 500 are not subtracted. Here is the same number using the subtraction rule:

$$1 \times 1000 + (1 \times 1000) - (1 \times 100) + 1 \times 50 + 3 \times 10 + (1 \times 5) - (1 \times 1) = 1984$$

$$\mathbf{M} + \mathbf{CM} + \mathbf{L} + \mathbf{XXX} + \mathbf{IV} = \mathbf{MCMLXXXIV}$$

Numbers, numbers...

The Romans also used [fractions](#). The most common [base](#) for fractions was 1/12, which in Latin is called *uncia* ([ounce](#)).

Fraction	Numeral	Name (nominative and genitive)	Meaning
1/12	·	<i>Uncia, unciae</i>	"Ounce"
2/12 = 1/6	·· or :	<i>Sextans, sextantis</i>	"Sixth"
3/12 = 1/4	··· or ∴	<i>Quadrans, quadrantis</i>	"Quarter"
4/12 = 1/3	···· or ∷	<i>Triens, trientis</i>	"Third"
5/12	····· or ∴∴	<i>Quincunx, quincuncis</i>	"Five-ounce" (<i>quinque unciae</i> → <i>quincunx</i>)
6/12 = 1/2	S	<i>Semis, semissis</i>	"Half"
7/12	S ·	<i>Septunx, septuncis</i>	"Seven-ounce" (<i>septem unciae</i> → <i>septunx</i>)
8/12 = 2/3	S ·· or S :	<i>Bes, bessis</i>	"Twice" (as in "twice a third")
9/12 = 3/4	S ··· or S ∴	<i>Dodrans, dodrantis</i> or <i>nonuncium, nonuncii</i>	"Less a quarter" (<i>de-quadrans</i> → <i>dodrans</i>) or "ninth ounce" (<i>nona uncia</i> → <i>nonuncium</i>)
10/12 = 5/6	S ···· or S ∴∴	<i>Dextans, dextantis</i> or <i>decunx, decuncis</i>	"Less a sixth" (<i>de-sextans</i> → <i>dextans</i>) or "ten ounces" (<i>decem unciae</i> → <i>decunx</i>)
11/12	S ····· or S ∴∴∴	<i>Deunx, deuncis</i>	"Less an ounce" (<i>de-uncia</i> → <i>deunx</i>)
12/12 = 1	I	<i>As, assis</i>	"Unit"

Numbers, numbers...

The simple but efficient ancient **Chinese numbering system**, which dates back to at least the 2nd millennium BCE, used small bamboo rods arranged to represent the numbers 1 to 9, which were then placed in columns representing units, tens, hundreds, thousands, etc. It was, therefore, a **decimal place value system**, very similar to the one we use today – indeed it was the first such number system, adopted by the Chinese over a thousand years before it was adopted in the West – and it made even quite complex calculations very quick and easy.



Example: 924



Written numbers, however, employed the slightly less efficient system of using a different symbol for tens, hundreds, thousands, etc. This was largely because there was no concept or symbol of zero, and it had the effect of limiting the usefulness of the written number in Chinese.

Numbers, numbers...

The **Hindu–Arabic numeral system**, sometimes known as **Hindu–Arabic numbers**, is the system of numbers used by almost all the world today.^[1] It is a positional notation system. This system has ten basic **symbols**, they are 0, 1, 2, 3, 4, 5, 6, 7, 8 and 9. It represent numbers in the decimal number system. The Hindu–Arabic numeral system was first developed by the **Hindus**. Later, It was introduced to the western world by the **Arabs**. The Hindu–Arabic numeral system is one of the few **numeral systems** that use a **place value** system. This system is now commonly used all over the world.

They originated in India in the 6th or 7th century and were introduced to Europe through the writings of Middle Eastern mathematicians, especially **al-Khwarizmi** and **al-Kindi**, about the 12th century. They are very different from previous methods of counting, such as the **abacus**, and paved the way for the development of algebra. In the past many other systems were used.^[2] **Fibonacci** popularized the **Hindu–Arabic numeral system** to the Western World.

Place value in **mathematics** is the value represented by a digit.

In the decimal system (which is base 10) with digits (0-9) the first column left of the decimal place is the units column, the next column to the left is the tens, the next is the hundreds. The first column right of the decimal place is the tenths column, the next is hundredths, the next is thousandths.

This can be illustrated as H T U . t h th

Numbers, numbers...

In [mathematics](#), a **rational number** is a [number](#) that can be expressed as the [quotient](#) or [fraction](#) $\frac{p}{q}$ of two [integers](#), a [numerator](#) p and a non-zero [denominator](#) q .^[1] For example, $\frac{-3}{7}$ is a rational number, as is every integer (e.g. $5 = \frac{5}{1}$). The [set](#) of all rational numbers, also referred to as "**the rationals**",^[2] the **field of rationals**^[3] or the **field of rational numbers** is usually denoted by a boldface **Q** (or [blackboard bold](#) $\mathbb{Q}$, Unicode U+1D410 **Q** MATHEMATICAL BOLD CAPITAL Q or U+211A **Q** DOUBLE-STRUCK CAPITAL Q);^[4] it was thus denoted in 1895 by [Giuseppe Peano](#) after [quoziente](#), Italian for "[quotient](#)",^[citation needed] and first appeared in Bourbaki's *Algèbre*.^[5]

The [decimal expansion](#) of a rational number either terminates after a finite number of [digits](#) (example: $\frac{3}{4} = 0.75$), or eventually begins to [repeat](#) the same finite [sequence](#) of digits over and over (example: $\frac{9}{44} = 0.20454545\dots$).^[6] Conversely, any repeating or terminating decimal represents a rational number. These statements are true in [base 10](#), and in every other integer [base](#) (for example, [binary](#) or [hexadecimal](#)).^[citation needed]

A [real number](#) that is not rational is called [irrational](#).^[5] Irrational numbers include $\sqrt{2}$, π , e , and ϕ . The [decimal expansion](#) of an irrational number continues without repeating. Since the set of rational numbers is [countable](#), and the set of real numbers is [uncountable](#), [almost all](#) real numbers are irrational.^[1]

Computers can only use integers and only integers up to a certain value. To represent non-integers, the computer must do some tricks with its integers to represent non-integers and it can't represent all non-integers exactly (or integers larger than a certain value). It can represent some numbers symbolically, to refer to numbers that cannot be otherwise represented by 32-bit or 64-bit integers.

So, **computers** work neither on **rational nor on irrational numbers**, they are designed to work on a finite set of numbers.

1. Describe and explain the pros and cons of fixed and floating number system, particularly the representable range and precision. Illustrate using example of 32-bit fixed point and the 32-bit floating point number in single precision IEEE754 format.

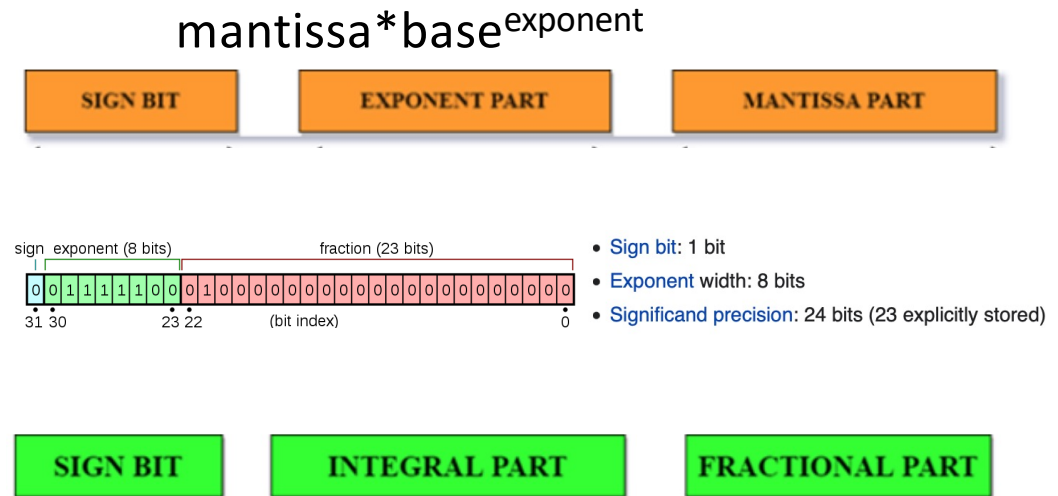
- Floating Point (IEEE754)

- Max Range $\approx 2 \cdot 2^{128}$ ($\sim 2^{128}$ to -2^{128}).
- Max Precision (near to zero) less than 2^{-126}

- Fixed Point

- Max Range (Radix right of LSB, unsigned) $\approx 2^{32}$
- Max Precision (Radix left of MSB) 2^{-32}

- Floating point yield a larger range and better precision at small numbers with the same number of bits representation.
- One usually needs the best precision when the numbers are small.
- However, Fixed point number has the advantage of having uniform precision across entire range.
- Floating point number's precision changes across the range and the very coarse precision at the two end of the range may not be desirable to the intended algorithm.



2. An array consisting of the length of 256 wires is given by $L[0], L[1], \dots, L[255]$. Assume that user are not allowed to test for any overflow during computation, describe a scheme to compute the average length of the 256 wires that will yield a result with the highest precision based on the following specifications:

- 16-bit registers are used for storing the data and result.
- Only Single-Precision (16-bit) and Fixed-Point arithmetic is used.
- Maximum possible length of each wire is 0x3FF and is an integer.

No coding is required, illustrate your answer in the form of a mathematical expression and justify your answer.

- Unsigned arithmetic used to maximise the range since the wire length cannot be negative
- 16-bit $\Rightarrow$ maximum 0xFFFF 65,535
- Max length of one wire = 0x3FF $\Rightarrow$ Max number of wire length that can be accumulated before the result will potentially overflow = 64 1,023
- Math Expression = 1,023*64=65,472

$$\{(L[0]+\dots+L[63])/64 + (L[64]+\dots+L[127])/64 + (L[128]+\dots+L[191])/64 + (L[192]+\dots+L[255])/64\}/4$$

Always perform Division last to avoid truncating the LSBs too early in the computation. But need to take note of overflow when using addition, subtraction and multiplication.

3. Consider a processor (not ARM processor) with 4 pipeline stages: Fetch Instruction (F), Decode (D), Execute (E) and Store (S). Assume that

- Branch target address is calculated at the execute stage
- Instruction length for every instruction is one word long
- Each pipeline stage take 1 cycle to complete
- Delay Branching is not enabled.

```

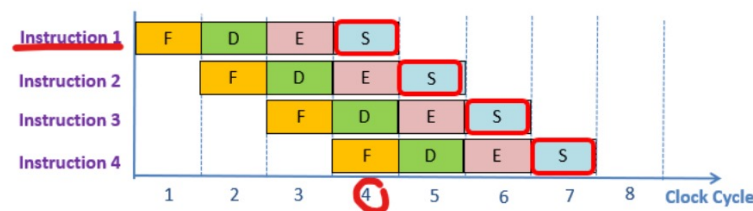
MOV R6, #0x900 ; I1
MOV R5, #0      ; I2
MOV R4, #0x800 ; I3
MOV R3, #0x300 ; I4
Loop LDR R0, [R3] ; I5
     LDR R1, [R4] ; I6
     ADD R2, R1, R0 ; I7
     ADD R5, R5, #1 ; I8
     ADD R4, R4, #1 ; I9
     ADD R3, R3, #1 ; I10
     CMP R5, #5     ; I11
     BNE Loop      ; I12
     ADD R4, R4, R2 ; I13
     STR R2, [R6]   ; I14
  
```

Figure 9.2

How many cycles does the code in Figure 9.2 take? You can ignore all pipeline conflicts you see in the code, just focus on the pipeline behaviour and execution cycles.

- I1 to I4: 7 cycles (I1 will take 4 cycles to pass through the pipeline)
- I5 to I12. For the first four iterations, each iteration takes (8+2) cycles. This includes 2 cycles discarded in each iteration.
- I5 to I12. Fifth iteration takes 8 cycles since the branch is not taken so no instructions are discarded.
- I13 to I14: 2 cycles.
- Total = $7 + (8+2)*4 + 8 + 2 = 57$ cycles

For a 4-stage pipeline, First instruction will only be completed at clock cycle 4.



Additional cycle comes from the 2-cycle Branch Delay in a 4-stage pipeline that resolves its branch target at the E stage (if the Branch is Taken).

- Branch statements in pipelining can lead to **Branch Delay** which cause significant performance loss.



- The **branch target** is only known after the **Execute** stage, but by this time, **Instructions I and J** have already been **fetched**.
- Instructions I and J will be **discarded**, resulting in **two-cycle branch delay**.
- The two slots that are discarded is known as the **Delay Slots**
- Instruction I and J is known as the **Delay Slot Instruction**

4. Consider a processor (not ARM processor) with 4 pipeline stages: Fetch Instruction (F), Decode (D), Execute (E) and Store (S). Assume that

- Branch target address is calculated at the execute stage
- Instruction length for every instruction is one word long
- Each pipeline stage takes 1 cycle to complete
- No Resource Conflicts
- Delayed Branching is enabled

Identify and describe ALL pipeline conflicts the code in Figure 9.3 has when executed in the pipeline processor above. Suggest workaround for pipeline conflicts identified.

	MOV	R6,	#0x900	;	I1
	MOV	R5,	#0	;	I2
	MOV	R4,	#0x800	;	I3
	MOV	R3,	#0x300	;	I4
Loop	LDR	R0,	[R3]	;	I5
	LDR	R1,	[R4]	;	I6
	ADD	R2,	R1, R0	;	I7
	ADD	R5,	R5, #1	;	I8
	CMP	R5,	#5	;	I9
	BNE	Loop		;	I10
	ADD	R4,	R4, #1	;	I11
	ADD	R3,	R3, #1	;	I12
	ADD	R4,	R4, R2	;	I13
	STR	R2,	[R6]	;	I14

- I4 and I5. Data dependency. R3 register value is used in I5 before I4 completes loading R3 register. Resolved by
 - swapping I3 and I4, OR swapping I5 and I6, OR
 - inserting a NOP instruction between I4 and I5
- I6 and I7. Data dependency. R1 register value is used in I7 before I6 completes loading R1 register. Resolved by
 - swapping I7 and I8, OR
 - inserting a NOP instruction between I6 and I7
- I8 and I9. Data dependency. R5 register value is used in I9 before I8 completes loading R5 register. Resolved by
 - swapping I7 and I8, OR
 - inserting a NOP instruction between I8 and I9
- I5 and I12. Data dependency. I11 and I12 are delay slot instructions. During the first four iterations of the loop, I5 uses R3 before I12 can load the latest increment to R3. Resolved by
 - swapping I11 and I12, and add a NOP before I13.
- No branch conflicts since delayed branching is used.
- No data dependency between I13 and I14 since I14 is just reading and not writing to R2

5. You have been tasked to write a program that calculates the actual time based on a counter that is incremented once every 0.10 seconds. For example, if the counter value is 3,600,000, you would expect the actual time to be 100 hours $((3,600,000 \times 0.1) / (60 \times 60))$.

Suppose you have decided to use a 24-bit fixed point representation as shown in Figure 9.4 to store the value of 0.10 seconds $(2^{-4} + 2^{-5} + 2^{-8} + 2^{-9} + 2^{-12} + 2^{-13} + \dots)$.

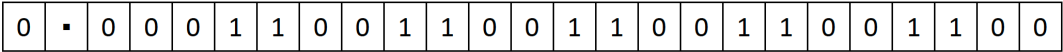


Figure 9.4 – Fixed point representation of 0.1010

- a. Approximate the round-off error (in decimal) of 0.10₁₀ due to the fixed-point representation.
- b. What is the effect of this round-off error on the time calculated if the counter value is 3,600,000?

a. The decimal value of the fixed point representation in Figure 9.4 is 0.0999999046325684₁₀.

Hence, the round-off error is approximately 0.000000095₁₀ (0.10₁₀ - 0.0999999046325684₁₀).

b. The time calculated will be off by approximately 0.34 seconds (3600000₁₀ x 0.000000095₁₀).

Note: This question is based on an actual example of disasters caused by bad numeric. The inaccurate calculation of time has contributed to fatalities when it was used for intercepting enemy missiles during Gulf War. While an error of 0.34 seconds may not seem like much, a missile travelling at 1,676 meters per second would have travelled approximately 500 meters in that time.

$$0.1 \times 2 = 0 + 0.2$$

$$0.2 \times 2 = 0 + 0.4$$

$$0.4 \times 2 = 0 + 0.8$$

$$0.8 \times 2 = 1 + 0.6$$

$$0.6 \times 2 = 1 + 0.2$$

$$0.2 \times 2 = 0 + 0.4$$

$$0.4 \times 2 = 0 + 0.8$$

$$0.8 \times 2 = 1 + 0.6$$

$$0.6 \times 2 = 1 + 0.2$$

$$0.2 \times 2 = 0 + 0.4$$

Step 1: Divide 0 by 2 keeping notice of the quotient and the remainder. Continue dividing the quotient by 2 until you get a quotient of zero.

Step 2: Then just write out the remainders in the reverse order to get the equivalent binary number.

0 / 2 = 0 with remainder 0

Here is the answer to 0 decimal to binary number:

0

For converting decimal fraction 0.1 to binary number, follow these steps:

Step 1: Multiply 0.1 by 2 keeping notice of the resulting integer and fractional part. Continue multiplying by 2 until you get a resulting fractional part equal to zero (we calculate upto ten digits).

Step 2: Then just write out the integer parts from the results of each multiplication to get equivalent binary number.